

Implementing Long-Term Semantic Memory for an Agent

What is Long-Term Semantic Memory?

Long-term semantic memory is implemented by storing important information as embeddings in a vector database and retrieving relevant memories through similarity search whenever the agent needs context.

The Problem

A user may state a preference today and ask a related question weeks later. Conversation history may be unavailable, so the agent needs persistent memory.

Step 1: Extract Important Information

Identify memory-worthy facts such as preferences, goals, skills, and stable facts. Example: 'I prefer Java for coding interviews.'

Step 2: Convert to Embeddings

Convert the memory into a vector representation using an embedding model.

Step 3: Store in a Vector Database

Store memory text, embedding vectors, and metadata in systems such as Chroma, FAISS, Pinecone, or Weaviate.

Step 4: Retrieve Relevant Memories

Embed the user's new query and perform similarity search to find the most relevant memories.

Step 5: Inject Memory into the Prompt

Include retrieved memories in the prompt so the LLM can personalize its response.

Architecture

User Message -> Memory Extractor -> Embedding Model -> Vector Store. Later: New Question -> Embedding Model -> Similarity Search -> Relevant Memories -> Prompt -> LLM.

LangGraph Implementation

Use retrieval nodes before the LLM and memory-storage nodes after the LLM. Retrieved memories are added to graph state before response generation.

Types of Semantic Memory

Preferences, skills, goals, and stable background information are commonly stored.

Production Improvements

Add memory scoring, deduplication, memory updates, and memory categorization for better quality.

How to Find User Preferences

The challenge is deciding what information is worth remembering.

What Counts as a Preference?

Likes/dislikes, choices, habits, goals, and stable facts. Example: 'I prefer Java', 'I usually study at night', 'I am preparing for SDE interviews'.

Rule-Based Extraction

Use keyword patterns such as 'I prefer', 'I like', 'I usually', or 'My goal is'. This is simple but limited.

LLM-Based Extraction

Use a memory extraction prompt to identify preferences, goals, skills, and stable facts, and return them in structured JSON.

Avoid Storing Everything

Use importance thresholds, deduplication, and retention policies. Store information that is likely to be useful again in the future.

Example

User says: 'I am preparing for SDE interviews', 'I prefer Java for DSA', 'I study at night'. These become stored memories and are later retrieved to personalize responses.

Interview Answer

Long-term semantic memory is implemented by extracting important facts, converting them into embeddings, storing them in a vector database, retrieving relevant memories through similarity search, and injecting those memories into prompts. Preference extraction is typically done using rule-based heuristics or LLM-based memory extraction pipelines.